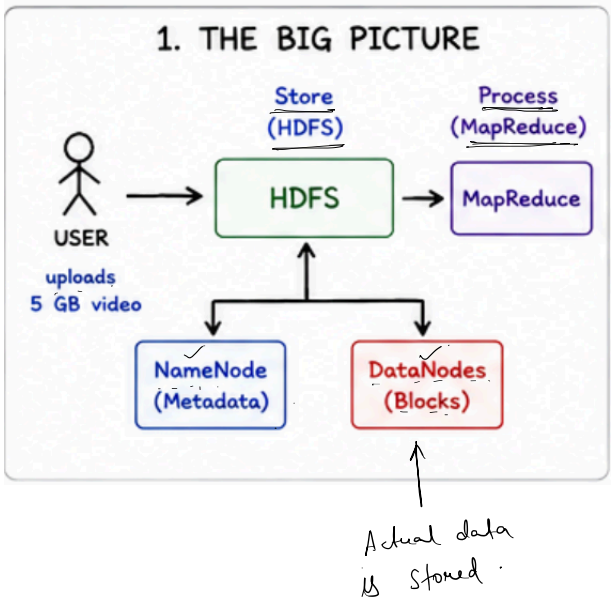
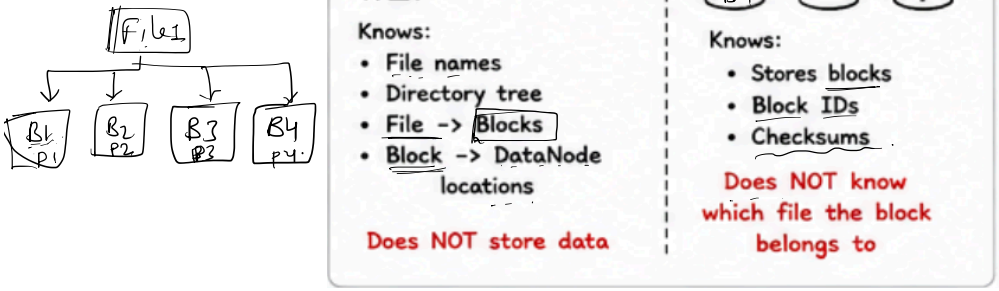
HDFS and MAP-REDUCE

Friday, 14 August 2026 1:32 AM

1) Hadoop
2) Map Reduce.

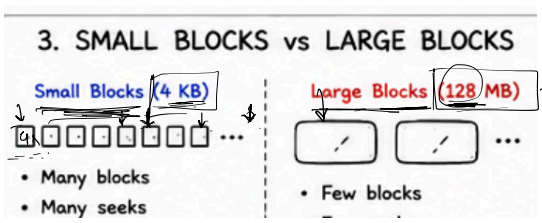
Hadoop Distributed File System (HDFS).





1 GB → small chunks.

1GB / 4KB → ~250,000

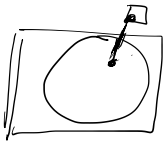


Try to access data:
- seek time.
- Read time.

1GB / 128MB → 8

- Slow
- Few seeks
- Fast

HDFS Goal:
Spend 99% time transferring data, only 1% seeking.



4. SEEK TIME vs TRANSFER TIME

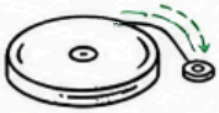
Seek Time (Slow)

Moving the arm to the right track (~10 ms)



Transfer Time (Fast)

Reading data once the arm is there (~100 MB/s)



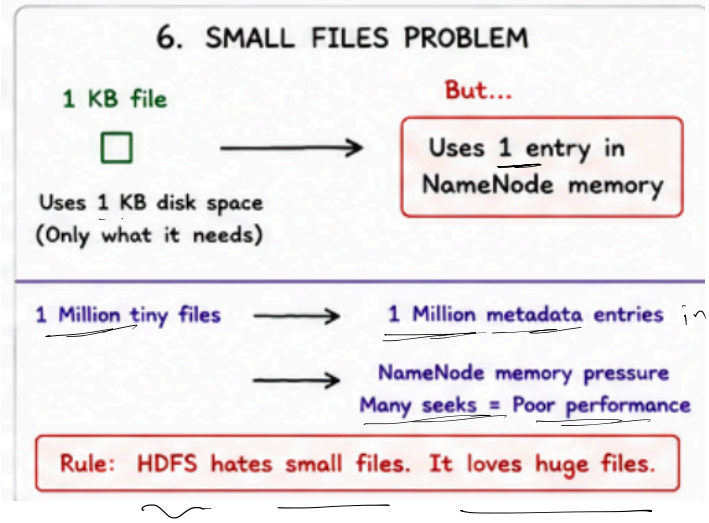
Seek = Finding the data
Transfer = Reading the data

5. 1 GB FILE – LET'S DO THE MATH!

Scenario	Laptop (4 KB Blocks)	HDFS (128 MB Blocks)
Number of Blocks	250,000	8
Seek Time (10ms each)	250,000 × 10ms = 2,500 sec (~41 min)	8 × 10ms = 0.08 sec
Transfer Time (@100 MB/s)	~10 sec	~10 sec
Total Time	~41 Minutes	~10.08 Seconds

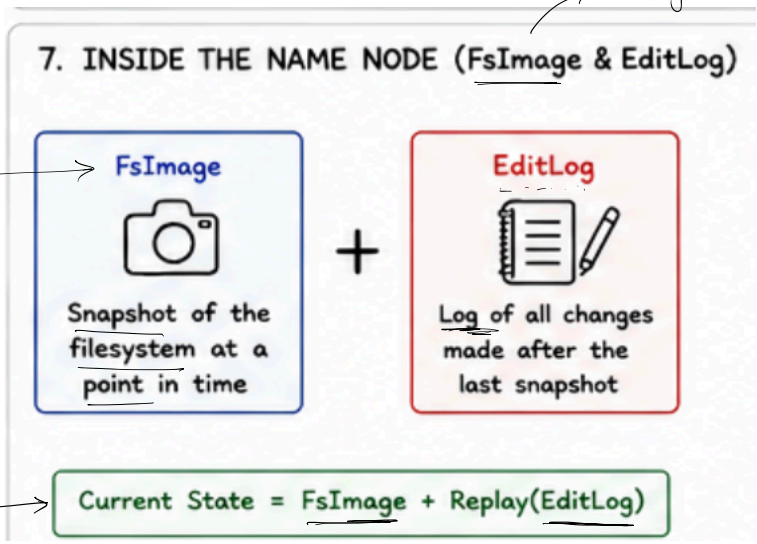
Conclusion: Large blocks drastically reduce seek overhead.

→ HDFS loves large datasets.

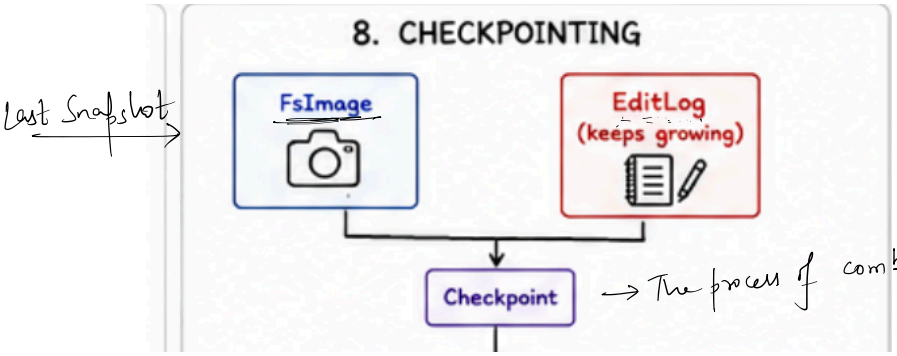


in Namenode.

File System Image.



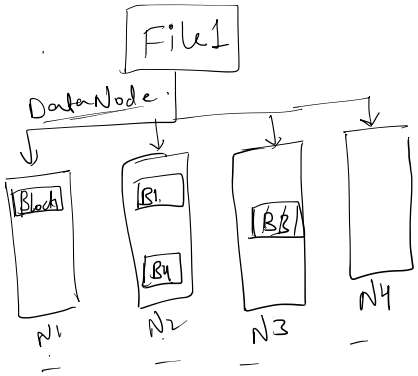
7 AM.



Last Snapshot

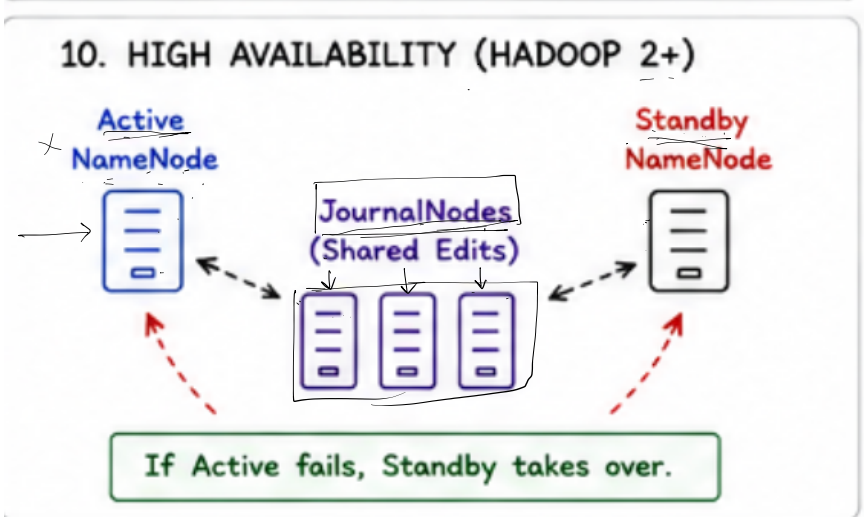
→ The process of combining FsImage + EditLog



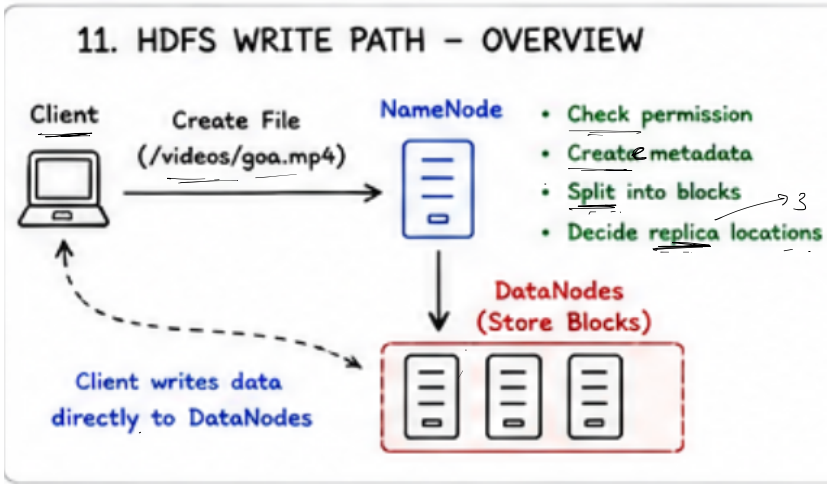


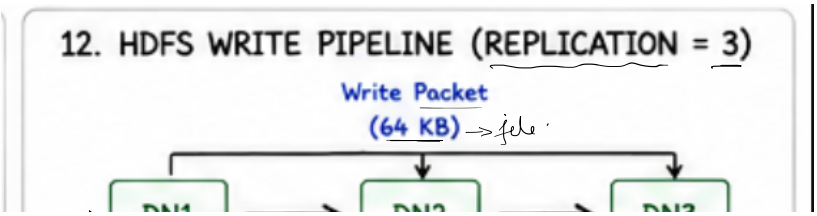
NameNode.
F1 / B1 → N1/Block1
F1 / B2, B4 → N2/B1 , N2/B4
F1 / B3 → N3/B3.

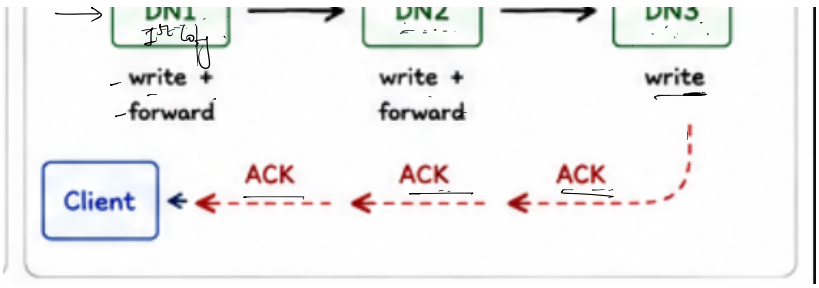
Editlog
Del F1/B3.

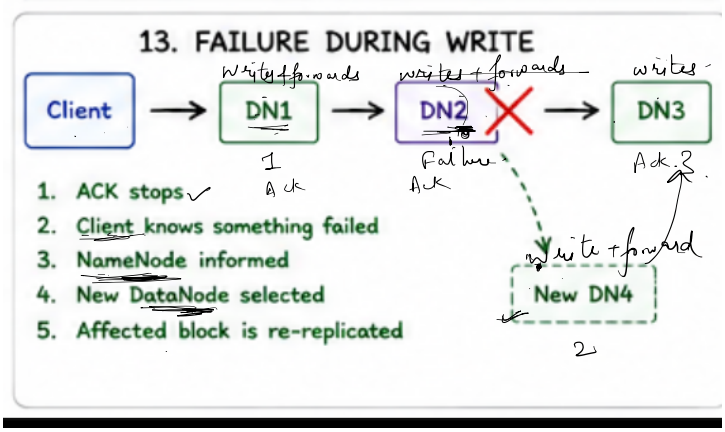


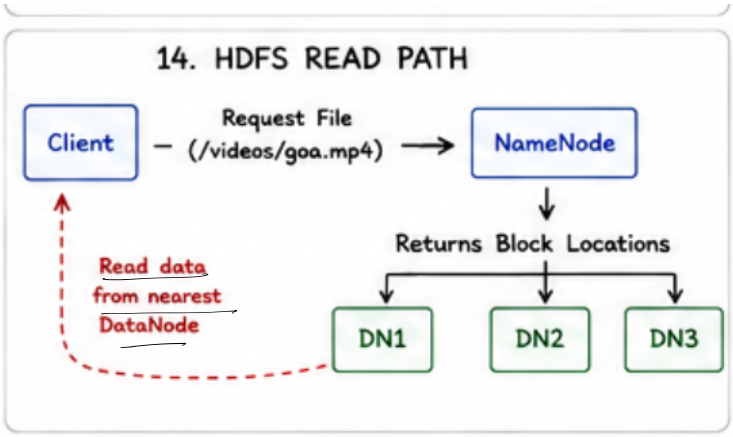
OLAP.

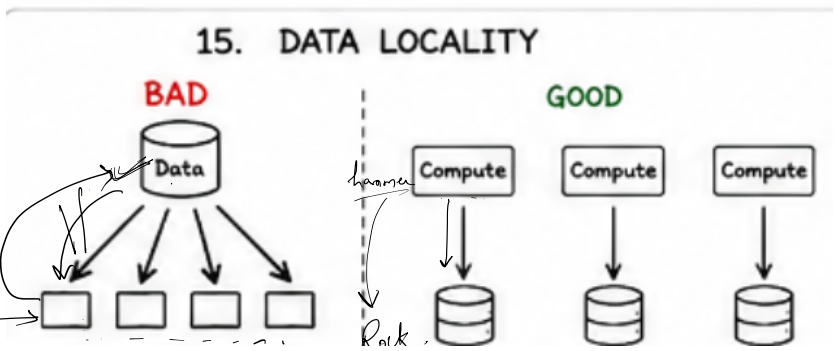




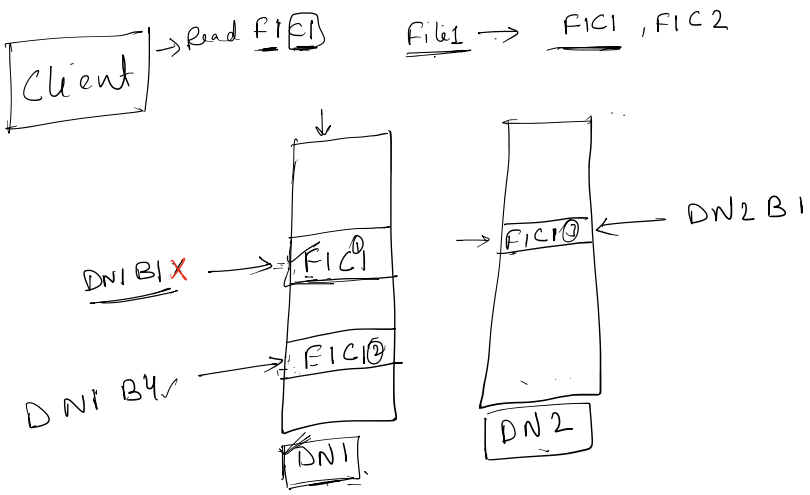

write +
forward
write +
forward
write
Client
ACK
ACK
ACK


13. FAILURE DURING WRITE
Client
DN1
DN2
DN3
New DN4
1. ACK stops
2. Client knows something failed
3. NameNode informed
4. New DataNode selected
5. Affected block is re-replicated

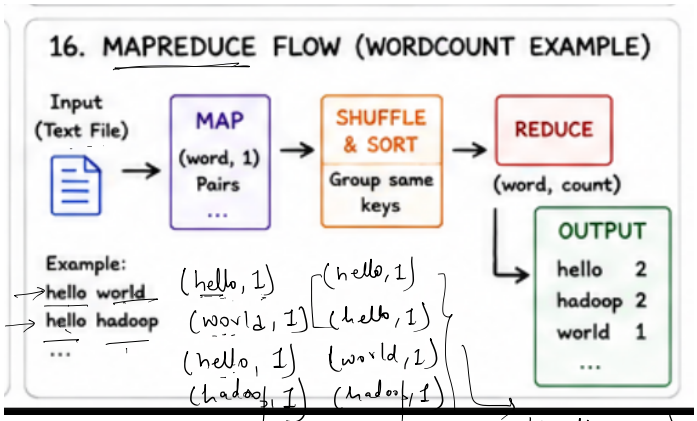

14. HDFS READ PATH
Client
Request File
(/videos/goa.mp4)
NameNode
Returns Block Locations
Read data
from nearest
DataNode
DN1
DN2
DN3


15. DATA LOCALITY
BAD
GOOD
Data
Compute
Compute
Compute

Moving data to compute (Expensive)	Moving compute to data (Cheap)



Google.



Functional Programming

map

reduce.

* Map → apply the same function to every element in the file/list
* Reduce → Combine results into a final answer.